Modern Cloud Computing

Lecture 20: ML in Cloud I

Chongzhou Fang

Rochester Institute of Technology

ML-as-a-Service (MLaaS)

Challenges:

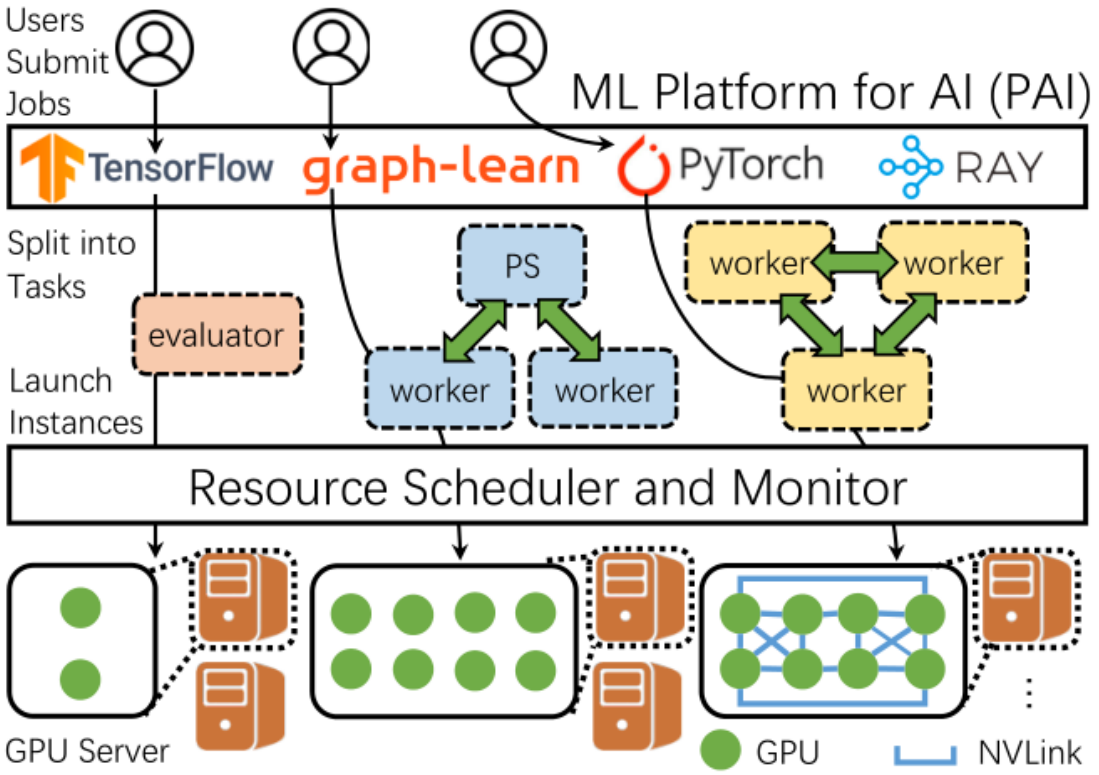
- **GPUs are expensive and, by default, indivisible** — one job per device
- **Distributed training needs gang scheduling** — all workers start together
- **Performance depends on user-chosen knobs** — batch size, learning rate, #. of GPUs
- **Hardware is heterogeneous** — T4, P100, V100, NVLink vs. PCIe

MLaaS in the Wild: Workload Analysis and Scheduling in Large-Scale Heterogeneous GPU Clusters

Weng, Xiao, Yu, Wang, Wang, He, Li, Zhang, Lin, Ding — NSDI 2022

Alibaba PAI: Platform for AI

- Users submit **jobs** in TensorFlow, PyTorch, Graph-Learn, RLlib, ...
- Job → **tasks** by role (parameter server, worker, evaluator) → **instances** in Docker containers
- Instances of a task are **gang-scheduled**; some need **GPU locality** (one machine, NVLink)
- **GPU sharing**: an instance may request a fraction of a GPU in $(0, 1)$
- The scheduler sees only containers (agnostic to ML semantics)



(Source: Weng et al., NSDI '22, Fig. 1)

The PAI Trace

Two months (Jul–Aug 2020), **6,742 GPUs**, **1,300+ users**, **1.2M tasks**, **7.5M instances**, training *and* inference. Publicly released in the Alibaba Cluster Trace Program.

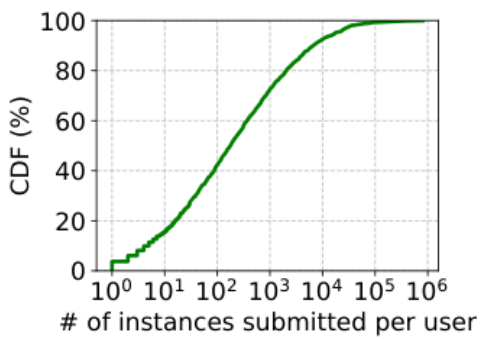
#GPUs / node	GPU type	vCPUs	Mem (GiB)	#Nodes
2	P100	64	512	798
2	T4	96	512	497
8	Misc (K40m, K80, M60)	96	512	280
8	V100M32 (NVLink)	96	384	135
8	V100 (NVLink)	96	512/384	104
0	— (CPU only)	96	512	83

Prior traces (e.g., Microsoft Philly) were smaller, training-only, and >82% single-GPU.

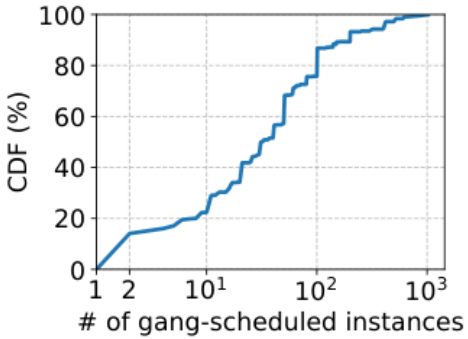
(Source: Weng et al., NSDI '22, Table 1)

Workload Features

- **Skewed users**: the top 5% of users submit ~77% of instances; the bottom 50% run <180 each
- **Gang scheduling is the norm**: ~85% of instances; 20% of those need >100 GPUs, some >1,000. Such tasks make up **79% of GPU demand**
  - Gang scheduling: a policy that launches all the instances (workers) of a distributed job simultaneously as one unit
- **Diurnal**: weekdays busier; midnight rush of many *small* tasks
- **Run-time spans 4 orders of magnitude**: median 23 min, P90 4.5 h (Philly: 26 min, 25 h)



(a) CDF of the number of instances run by a user.

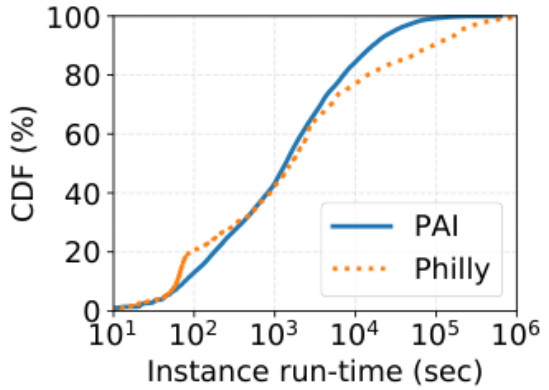


(b) CDF of gang-scheduled task instances.

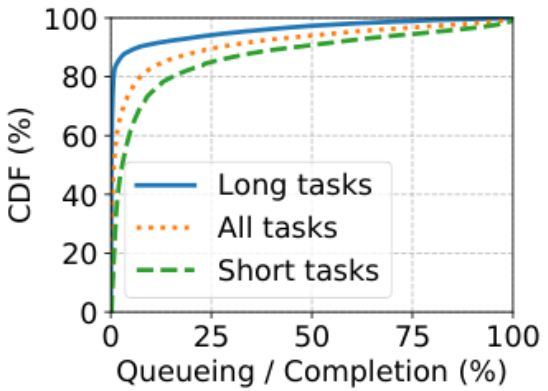
(Source: Weng et al., NSDI '22, Fig. 2–4)

Queueing Delays

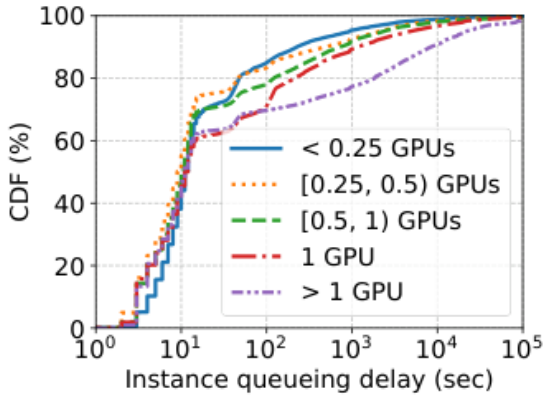
- **Short instances wait disproportionately**: ~9% of short-running instances spend >50% of their completion time queued (3% for long ones)
  - head-of-line blocking
- **Fractional-GPU requests schedule fast**: P90 delay **497 s** for < 1 GPU vs. **1,150 s** for 1 GPU and **8,286 s** for > 1 GPU
- **High-end GPUs are scarce**: V100 instances wait a median of 113 s and a P90 of **13,709 s**; low-end Misc GPUs: 11 s / 360 s



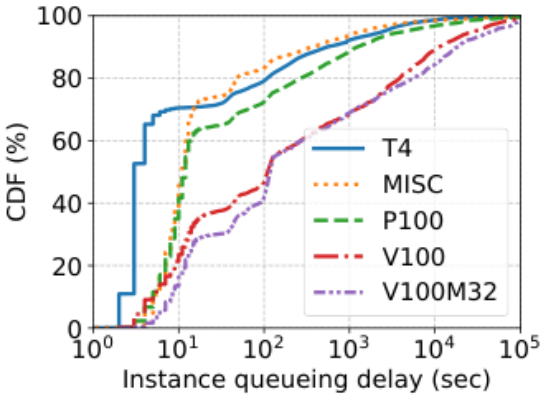
(a) CDF of instance run-time.



(b) CDF of normalized instance queueing delays.



(c) CDF of queueing delays w.r.t. GPU requests per instance.



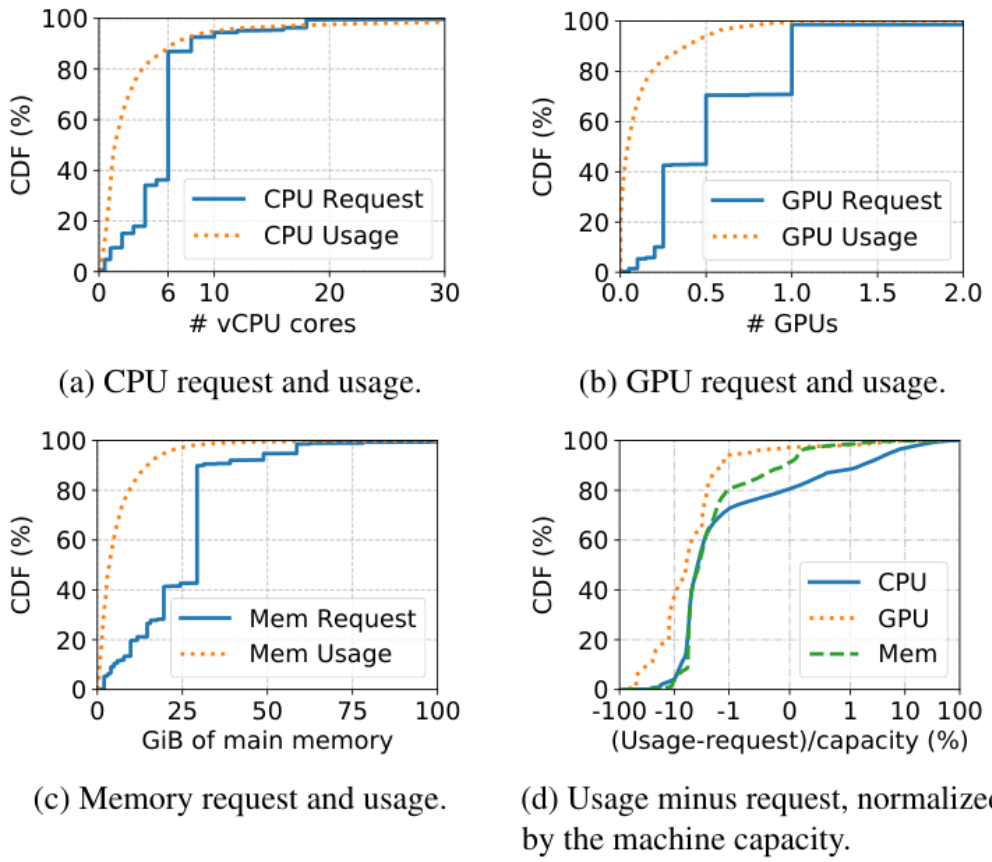
(d) CDF of queueing delays w.r.t. GPU types.

(Source: Weng et al., NSDI '22, Fig. 4)

User Requests

	Median request	Median usage
vCPU cores	6	**1.4**
GPUs	0.5	**0.042**
Memory	29 GiB	**3.5 GiB**

- Heavy-tailed: P95 asks for 12 vCPUs, 1 GPU, 59 GiB
- **18% of instances barely touch the GPU**
  - parameter servers, data fetching, preprocessing
- Instances can borrow *spare* host resources
  - **19% overuse CPU**, only 3% GPU, 9% memory



(a) CPU request and usage. (b) GPU request and usage.

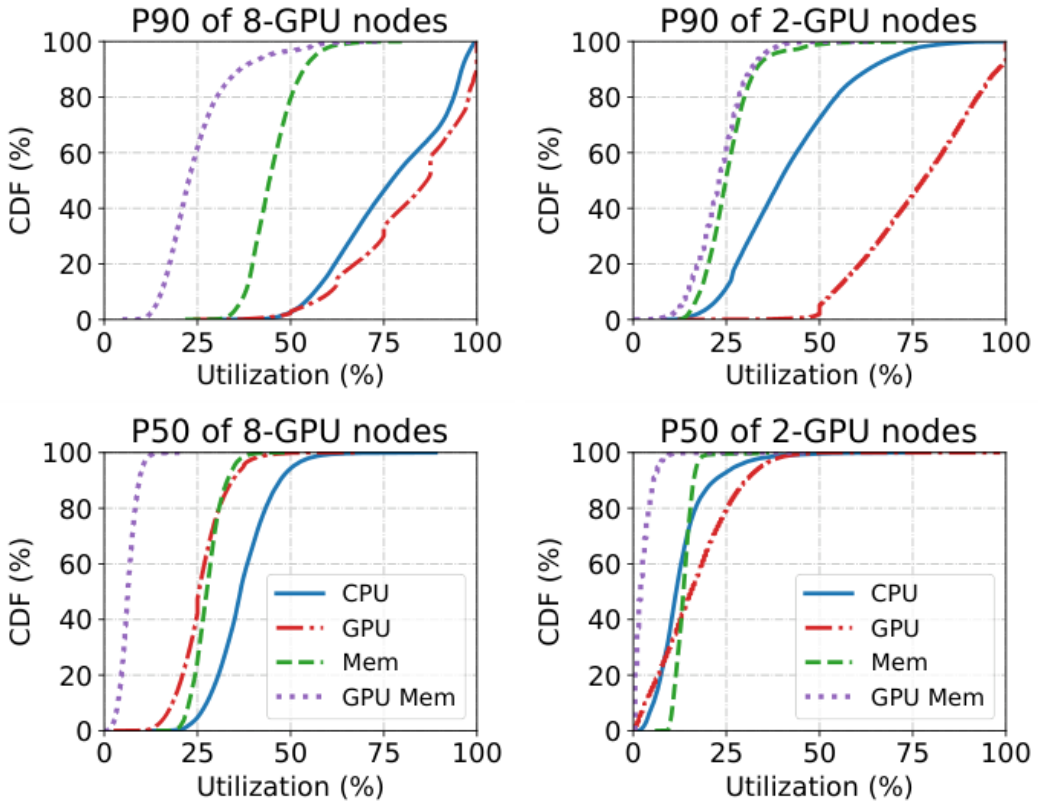
(c) Memory request and usage. (d) Usage minus request, normalized by the machine capacity.

(Source: Weng et al., NSDI '22, Fig. 5)

Machine-Level Utilization

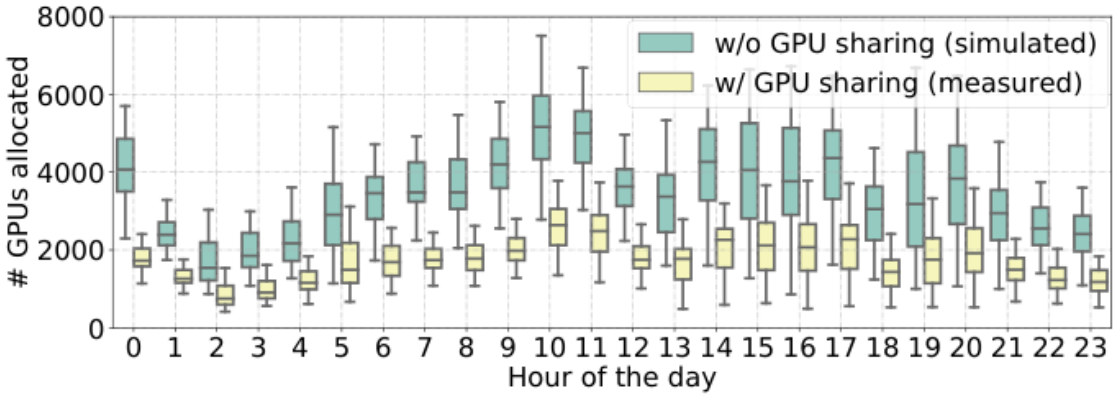
- **8-GPU nodes**: average P90 utilization **82% GPU / 77% CPU**
- **2-GPU nodes**: P90 GPU 77%, but P90 CPU only **42%** — 32–48 vCPUs per GPU is too many
- Main and GPU memory P90 stay **below 60%**
  - tasks are not memory-bound
- GPU utilization is highly variable (bursty workloads + a packing-first scheduler)
- Network and I/O usages are low
  - P95 ingress is 34-54% of guaranteed network bandwidth
  - CPUs are mostly busy processing data instead of waiting for I/O completion

(Source: Weng et al., NSDI '22, Fig. 6–7)



Opportunity 1: GPU Sharing

- **Mechanism**: GPU memory is space-multiplexed (guaranteed share); SMs are time-multiplexed (no isolation)
- **Savings**: replaying the trace *without* sharing would need **2× the GPUs** on average
- **Contention**: GPUs are ≥95% busy only **7%** of the time, and only **4.5%** of those GPUs host ≥2 instances
  - Sharing does not cause severe contention

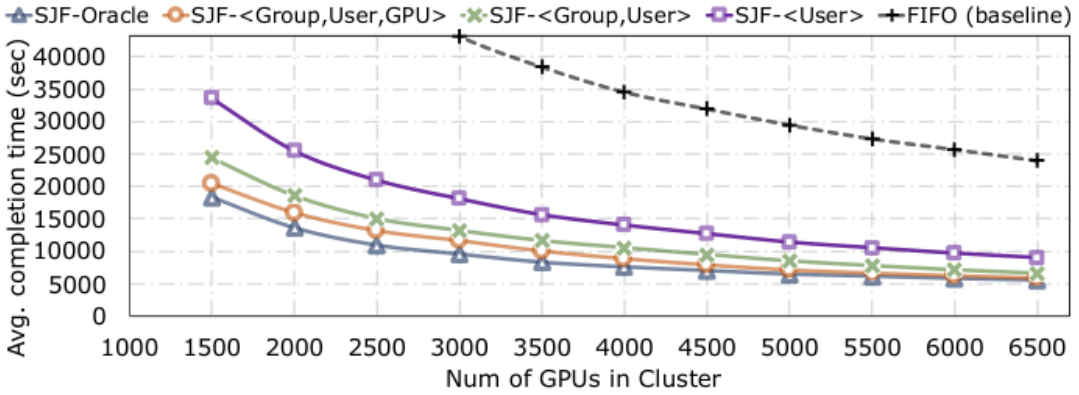


(Source: Weng et al., NSDI '22, Fig. 8–9)

Opportunity 2: Predictable Recurring Tasks

- The scheduler cannot see training progress
  - **65% of tasks recur ≥5 times**
- Hash meta-information (entry script, CLI args, data sources/sinks) into a **Group** tag
- CART regressor on `<Group, User, GPU Request>`: **78% of instances predicted instance duration within 25% error**
- Trace-driven simulation: Shortest-job-first (SJF) with predicted durations
  - cuts average completion time by **63–77%** vs. FIFO
  - close to SJF-Oracle (SJF based on real measured duration)

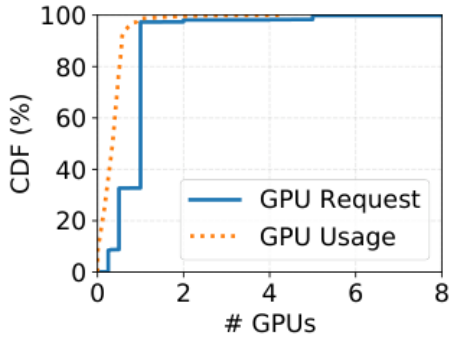
(Source: Weng et al., NSDI '22, Fig. 10, 12, 13)



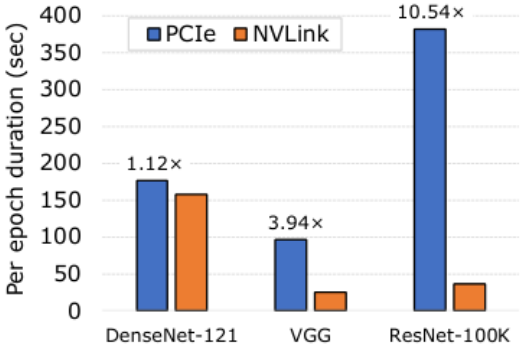
Case Study: High-GPU Tasks

A small set of business-critical, user-facing tasks:

- **NLP** (BERT, ALBERT, XLNet) ≈ 6.4% of tasks; 73% need GPUs with **≥16 GiB**; 40% request >1 GPU
- **ResNet-100k** (100k-class softmax): gradient exchange dominates → **GPU locality** matters. NVLink is **10.5×** faster per epoch than PCIe (VGG 3.9×, DenseNet-121 1.1×)
- Full GPUs + specific hardware → hard to place



(a) CDF of GPU requests and usages of NLP task instances.

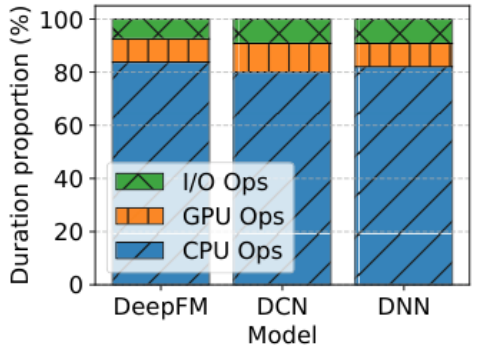


(b) Per-epoch duration of 3 classification models trained in 8-GPU machines with and without NVLink.

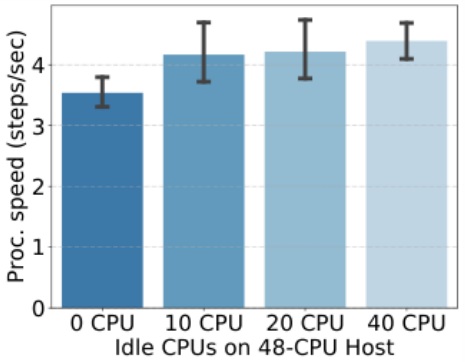
(Source: Weng et al., NSDI '22, Fig. 14)

Case Study: Low-GPU Tasks

- **Click-Through Rate (CTR) prediction** (≈6.7% of tasks, 75% inference):
  - >75% of instances use **<0.1 GPU**
  - **~80%** of run time is CPU ops (data fetching, etc.)
  - CPU contention slows down these tasks
- **GNN training**: CPU is more heavily utilized than GPU
  - graph preprocessing on CPU is **30–90%** of each iteration
- **RL training**: CPU heavy



(a) Duration breakdown of CTR prediction instances.



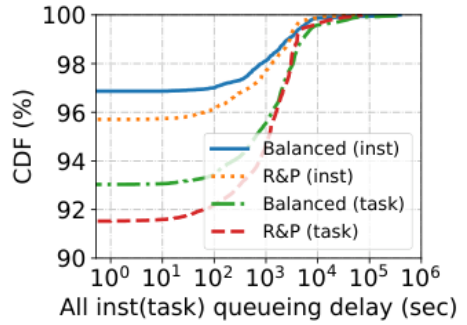
(b) DeepFM training instances interfered by the co-located load.

(Source: Weng et al., NSDI '22, Fig. 15–18)

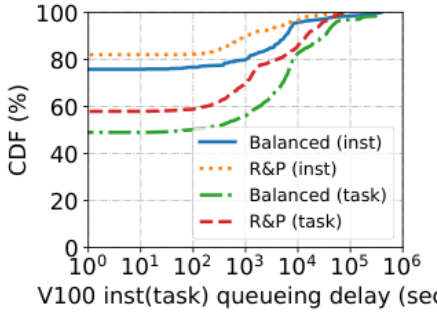
Deployed Policy: Reserving-and-Packing

- A **computation-efficiency** model (parallelism, model type, embedding size, history) splits tasks into *high-GPU* vs. *low-GPU*
- **Reserve** V100/V100M32 (NVLink) nodes for high-GPU tasks; **pack** the rest onto T4/Misc GPUs with GPU sharing
- Ordered allocation plans with timeouts
  - Similar to delay scheduling: first try high end GPUs, then lower-end GPUs
- **load balancing**: Priortize instance scheduling to machines with low allocation rate
- Simulation (100k tasks, 500k instances): average queueing delay **–45%** overall and **–68%** for V100 tasks vs. pure load balancing

(Source: Weng et al., NSDI '22, Fig. 19)



(a) Queueing delays of all instances and tasks.



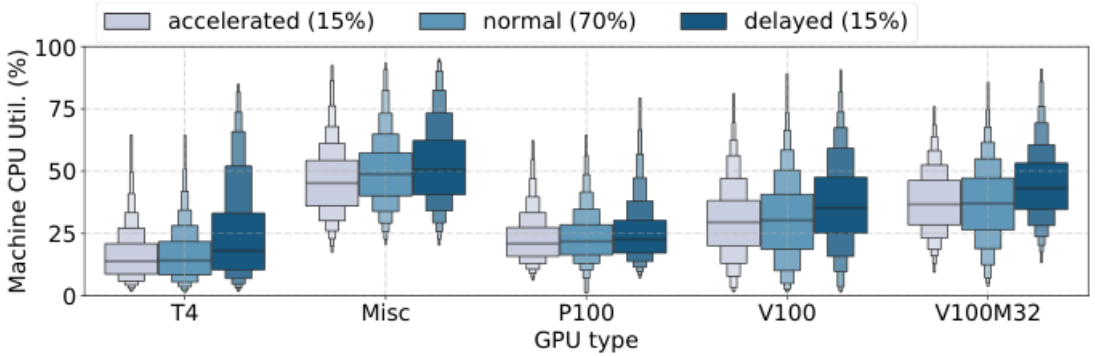
(b) Queueing delays of V100 instances and tasks.

Open Challenges

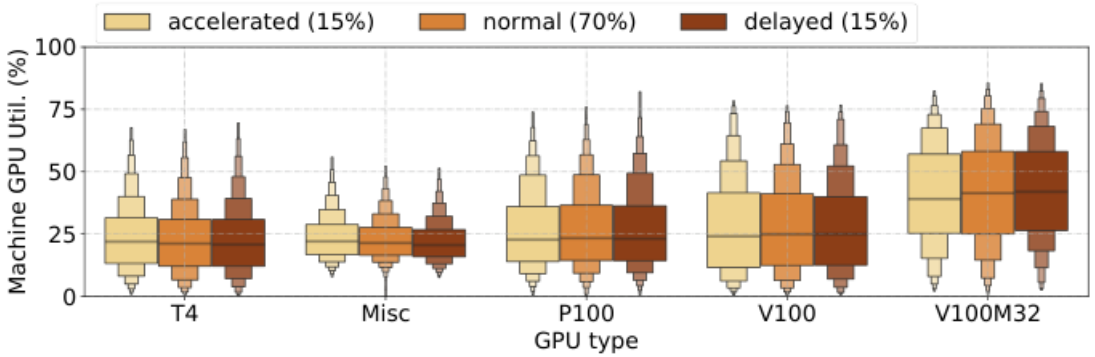
- **Provisioning mismatch** (vCPUs per GPU)

	All nodes	8-GPU nodes	2-GPU nodes
Machine specs	23.2	12.0	38.1
Instance requests	21.4	**22.8**	**18.1**

- **Overcrowded weak-GPU nodes**
- **Under-loaded high-end nodes**
- **CPU can be the real bottleneck**
  - Task scheduling should consider CPU contention



(a) CPU utilization of machines with various GPU types.



(b) GPU utilization of machines with various GPU types.

(Source: Weng et al., NSDI '22, Table 2, Fig. 21)